

Process Verification

Final Project

Quentin Meneghini

20th May 2026

Facultat d'Informàtica de Barcelona

Contents

1	Design Overview	1
1.1	Overview	1
1.2	Parameters	2
1.3	Top-Level Interface	2
1.4	Functional Description	3
2	Test Plan	4
2.1	Features Under Test	4
2.2	Tool for Testing	4
2.3	Traceability Matrix	5
2.4	Coverage	6
2.5	Stimulus	7
A	Appendix Chapter	8
A.1	Use of tools	8
A.2	Use of time	8
A.3	Use of AI	8

Chapter 1

Design Overview

1.1 Overview

The `gcd` module computes the Greatest Common Divisor of two unsigned integers using the subtractive Euclidean algorithm. It processes one input pair at a time and communicates through two independent ready/valid handshake channels.

Mathematical Definition

For unsigned operands A and B :

$$\text{gcd}(A, 0) = A$$

$$\text{gcd}(0, B) = B$$

$$\text{gcd}(0, 0) = 0 \quad (\text{covers both above})$$

$$\text{gcd}(A, A) = A$$

$$\text{gcd}(A, B) = \text{gcd}(A - B, B) \quad \text{when } A > B$$

$$\text{gcd}(A, B) = \text{gcd}(A, B - A) \quad \text{when } B > A$$

Operation Sequence

1. **IDLE:** The module waits in the IDLE state, asserting `in_ready = 1`.
2. **Handshake & RUN:** On the input handshake, operands are latched. The result is determined immediately for zero or equal operands; otherwise, iterative subtraction begins

in the `RUN` state.

3. **DONE:** When convergence is reached, the result is latched into the output register and the module enters the `DONE` state, asserting `out_valid = 1`.
4. **Return:** After the output handshake completes, the module returns to `IDLE`.

1.2 Parameters

Parameter	Default	Description
<code>WIDTH</code>	16 (must be ≥ 1)	Width of the data operands

Table 1.1: Module Parameters

1.3 Top-Level Interface

Clock and Reset

Signal	Direction	Width	Description
<code>clk</code>	Input	1	Clock signal
<code>rst_n</code>	Input	1	Active-low sync reset

Input Channel

Signal	Direction	Width	Description
<code>in_valid</code>	Input	1	Asserted by the producer when <code>a_in</code> and <code>b_in</code> hold valid data
<code>in_ready</code>	Output	1	Asserted by the module when it can accept a new transaction. High only in the <code>IDLE</code> state.
<code>a_in</code>	Input	<code>WIDTH</code>	First operand
<code>b_in</code>	Input	<code>WIDTH</code>	Second operand

Output Channel

Signal	Direction	Width	Description
<code>out_valid</code>	Output	1	Asserted by the module when <code>gcd_out</code> holds a valid result. High for the entire <code>DONE</code> state.
<code>out_ready</code>	Input	1	Asserted by the consumer when it is ready to accept the result.
<code>gcd_out</code>	Output	<code>WIDTH</code>	GCD result. Valid and stable for the entire period that <code>out_valid</code> is asserted.

1.4 Functional Description

The behavior of the `gcd` module is driven by a Finite State Machine (FSM) comprising three states:

FSM States

State	<code>in_ready</code>	<code>out_valid</code>	Description
IDLE	1	0	Waiting for input. Ready to accept a new transaction.
RUN	0	0	Iterative subtraction in progress. No new input accepted.
DONE	0	1	Result ready. Waiting for the consumer to accept it.

Algorithm Execution (RUN State)

Each clock cycle while in the RUN state, the working registers are updated combinatorially for the next cycle:

- If `a_reg > b_reg`: `a_next = a_reg - b_reg` and `b_next = b_reg`.
- Else if `b_reg > a_reg`: `b_next = b_reg - a_reg` and `a_next = a_reg`.

State Transitions

IDLE → DONE: Triggered by an input handshake **AND** (`a_in == 0 OR b_in == 0 OR a_in == b_in`). The result is known immediately, making this a 1-cycle transition.

IDLE → RUN: Triggered by an input handshake **AND** (`a_in != 0 AND b_in != 0 AND a_in != b_in`).

RUN → DONE: Triggered when the next-cycle values of the working registers are equal (`a_next == b_next`). This evaluates against the combinatorial *next-cycle* values, firing on the same clock edge as the final subtraction step.

DONE → IDLE: Triggered by a successful output handshake (`out_valid == 1 AND out_ready == 1`).

Chapter 2

Test Plan

2.1 Features Under Test

The features to be tested are divided into three groups. The idea is to test both normal execution and edge cases for each group.

- **[C] Control dependent (Consumer/Producer)** Reset behavior, Input ready/valid handshake/stall, Output ready/valid handshake/stall.
- **[D] Data dependent (Input/output)** edge cases (primes/coprimers, multiples, min value, max value), early exits (zero operands, equal operands), and correct output.
- **[S] State machine** Transitions between IDLE, RUN, and DONE, along with their cycle timings.

2.2 Tool for Testing

To test these features we will use the following tools:

- **[F] FORMAL:** here we will verify the logic from inside the dut. The analysis will touch the expected control output and the FSM states, hold, and transitions.
- **[U] UVM:** here we will verify the logic as a black box through stimulus. Using a golden model we will verify the correctness of the data.
- **[H] HARDWARE:** here we will verify the hardware timing and correct timings. Assertions based approach is used in a formal model.

2.3 Traceability Matrix

Here we introduce all the requirements extracted from the specification. It is divided in the previously presented features and sub-features accompanied by the tool used to verify it.

Req ID	Feature	Sub-Feature	Description	Tool
REQ_000	C	Reset	Correct init and graceful recover from reset mid-transaction	F/U
REQ_001	C	Input accept	Producer <code>in_valid</code> is high and DUT <code>in_ready</code> is high accepts input	F/U
REQ_002	C	Input stall	DUT <code>in_ready</code> is low and producer <code>in_valid</code> is high	H/F
REQ_003	C	Output accept	Consumer <code>out_ready</code> is high and DUT <code>out_valid</code> is high accepts output	H/F
REQ_004	C	Output stall	Consumer <code>out_ready</code> is low and DUT <code>out_valid</code> is high	H/F
REQ_005	C	Output stability	<code>gcd_out</code> is stable while <code>out_valid</code> is high	H/F
REQ_006	D	All operands	Correct <code>gcd_out</code> for general operands	U
REQ_007	D	Edge cases	Correct <code>gcd_out</code> for primes/coprimes, multiples, and min/max values	U
REQ_008	D	Zero operands	Correct <code>gcd_out</code> for $a = 0$ and/or $b = 0$, early exit	U
REQ_009	D	Equal operands	Correct <code>gcd_out</code> for $a = b$, early exit	U
REQ_010	S	States	FSM state is either <code>IDLE</code> , <code>RUN</code> , or <code>DONE</code>	F
REQ_011	S	State hold	FSM holds its state strictly for the necessary duration	F
REQ_012	S	Transitions	FSM transitions following the specification	F
REQ_013	S	Assigned signal	FSM determines output signals (<code>in_valid</code> , <code>out_ready</code>)	F

This second table is meant to associate each tool and relative test to the requirement it tries to verify. A description explains what the test is actually looking for.

Tool	Test name	Description	REQ ID
F	inputs	Assume a compliant producer that holds <code>in_valid</code> and data stable during stalls	REQ_002
F	<code>_reset</code>	Prove DUT initializes to <code>IDLE</code> and clears all registers on reset	REQ_000
F	<code>_state</code>	Prove FSM strictly operates within valid, mutually exclusive states	REQ_010
F	<code>idle_idle</code> , <code>run_run</code> , <code>done_done</code>	Prove FSM correctly holds current state when transition conditions are not met, including output stalls	REQ_004, REQ_011
F	<code>idle_done</code> , <code>idle_run</code> , <code>run_done</code> , <code>done_idle</code>	Prove FSM accurately transitions between states based on handshakes and data path evaluations	REQ_001, REQ_003, REQ_012
F	<code>_in_ready</code> , <code>_out_valid</code>	Prove control output signals are strictly tied to their respective FSM states	REQ_013
F	outputs	Prove <code>gcd_out</code> stability while <code>out_valid</code> is asserted	REQ_005
H	<code>chk_out_valid_no_drop</code>	Check that the DUT does not drop <code>out_valid</code> while waiting for the consumer <code>out_ready</code> to assert	REQ_004
H	<code>chk_out_data_stable</code>	Check that the DUT holds <code>gcd_out</code> stable while <code>out_valid</code> is asserted	REQ_005
H	<code>chk_reset</code>	Check that in case of reset the DUT clear all of its outputs	REQ_000
H	<code>chk_early_exit</code>	Check that the DUT skips to the <code>DONE</code> state in case of early exit	REQ_008, REQ_009
U	<code>scb.write_rst</code>	Verify that the golden model resets its internal state mid-transaction	REQ_000
U	<code>scb.write_in</code> , <code>scb.write_out</code>	Verify that the golden model output is equal to the DUT <code>gcd_out</code>	REQ_006, REQ_007, REQ_008, REQ_009

2.4 Coverage

The target coverage is just a measure of what we would like to tackle in this test plan for each area of the dut. In practice the objectives are a complete (100%) coverage of the functional description and possibly a high (<90%) code coverage. This excludes toggle as it is dependant of the width of the data.

2.5 Stimulus

The stimulus is divided into multiple tests that tackle different scenarios.

- **gcd_bringup_test**: Test the basic functionality of the verification system as a whole
- **gcd_det_test**: Test the edge cases and tries to obtain the maximum coverage with the minimal number of tests
- **gcd_rnd_test**: Test the DUT with random instructions to find edge cases that have not been tested otherwise

Appendix A

Appendix Chapter

A.1 Use of tools

- Visual Studio Code: systemverilog IDE.
- QuestaSim: CLI tester.
- GTKwave: wave visualizer.

A.2 Use of time

A total of 4 h distributed as follows.

- Understanding the problem: 1h.
- Report: 1h.

A.3 Use of AI

- Gemini (Pro): sentence enhancing, bug fixer